
Chapter 21. Database Roles

PostgreSQL manages database access permissions using the concept of *roles*. A role can be thought of as either a database user, or a group of database users, depending on how the role is set up. Roles can own database objects (for example, tables and functions) and can assign privileges on those objects to other roles to control who has access to which objects. Furthermore, it is possible to grant *membership* in a role to another role, thus allowing the member role to use privileges assigned to another role.

The concept of roles subsumes the concepts of “users” and “groups”. In PostgreSQL versions before 8.1, users and groups were distinct kinds of entities, but now there are only roles. Any role can act as a user, a group, or both.

This chapter describes how to create and manage roles. More information about the effects of role privileges on various database objects can be found in Section 5.8.

21.1. Database Roles

Database roles are conceptually completely separate from operating system users. In practice it might be convenient to maintain a correspondence, but this is not required. Database roles are global across a database cluster installation (and not per individual database). To create a role use the **CREATE ROLE** SQL command:

```
CREATE ROLE name;
```

name follows the rules for SQL identifiers: either unadorned without special characters, or double-quoted. (In practice, you will usually want to add additional options, such as **LOGIN**, to the command. More details appear below.) To remove an existing role, use the analogous **DROP ROLE** command:

```
DROP ROLE name;
```

For convenience, the programs `createuser` and `dropuser` are provided as wrappers around these SQL commands that can be called from the shell command line:

```
createuser name  
dropuser name
```

To determine the set of existing roles, examine the `pg_roles` system catalog, for example:

```
SELECT rolname FROM pg_roles;
```

or to see just those capable of logging in:

```
SELECT rolname FROM pg_roles WHERE rolcanlogin;
```

The `psql` program's `\du` meta-command is also useful for listing the existing roles.

In order to bootstrap the database system, a freshly initialized system always contains one predefined login-capable role. This role is always a “superuser”, and it will have the same name as the operating system user that initialized the database cluster with `initdb` unless a different name is specified. This role is often named `postgres`. In order to create more roles you first have to connect as this initial role.

Every connection to the database server is made using the name of some particular role, and this role determines the initial access privileges for commands issued in that connection. The role name to use for a particular database connection is indicated by the client that is initiating the connection request in an application-specific fashion. For example, the `psql` program uses the `-U` command line option to indicate the role to connect as. Many applications assume the name of the current operating system user by default (including `createuser` and `psql`). Therefore it is often convenient to maintain a naming correspondence between roles and operating system users.

The set of database roles a given client connection can connect as is determined by the client authentication setup, as explained in Chapter 20. (Thus, a client is not limited to connect as the role matching its operating system user, just as a person's login name need not match his or her real name.) Since the role identity determines the set of privileges available to a connected client, it is important to carefully configure privileges when setting up a multiuser environment.

21.2. Role Attributes

A database role can have a number of attributes that define its privileges and interact with the client authentication system.

login privilege

Only roles that have the `LOGIN` attribute can be used as the initial role name for a database connection. A role with the `LOGIN` attribute can be considered the same as a “database user”. To create a role with login privilege, use either:

```
CREATE ROLE name LOGIN;  
CREATE USER name;
```

(`CREATE USER` is equivalent to `CREATE ROLE` except that `CREATE USER` includes `LOGIN` by default, while `CREATE ROLE` does not.)

superuser status

A database superuser bypasses all permission checks, except the right to log in. This is a dangerous privilege and should not be used carelessly; it is best to do most of your work as a role that is not a superuser. To create a new database superuser, use `CREATE ROLE name SUPERUSER`. You must do this as a role that is already a superuser.

database creation

A role must be explicitly given permission to create databases (except for superusers, since those bypass all permission checks). To create such a role, use `CREATE ROLE name CREATEDB`.

role creation

A role must be explicitly given permission to create more roles (except for superusers, since those bypass all permission checks). To create such a role, use `CREATE ROLE name CREATEROLE`. A role with `CREATEROLE` privilege can alter and drop roles which have been granted to the `CREATEROLE` user with the `ADMIN` option. Such a grant occurs automatically when a `CREATEROLE` user that is not a superuser creates a new role, so that by default, a `CREATEROLE` user can alter and drop the roles which they have created. Altering a role includes most changes that can be made using `ALTER ROLE`, including, for example, changing passwords. It also includes modifications to a role that can be made using the `COMMENT` and `SECURITY LABEL` commands.

However, `CREATEROLE` does not convey the ability to create `SUPERUSER` roles, nor does it convey any power over `SUPERUSER` roles that already exist. Furthermore, `CREATEROLE` does not convey the power to create `REPLICATION` users, nor the ability to grant or revoke the `REPLICATION` privilege, nor the ability to modify the role properties of such users. However, it

does allow `ALTER ROLE ... SET` and `ALTER ROLE ... RENAME` to be used on `REPLICATION` roles, as well as the use of `COMMENT ON ROLE`, `SECURITY LABEL ON ROLE`, and `DROP ROLE`. Finally, `CREATEROLE` does not confer the ability to grant or revoke the `BYPASSRLS` privilege.

initiating replication

A role must explicitly be given permission to initiate streaming replication (except for superusers, since those bypass all permission checks). A role used for streaming replication must have `LOGIN` permission as well. To create such a role, use `CREATE ROLE name REPLICATION LOGIN`.

password

A password is only significant if the client authentication method requires the user to supply a password when connecting to the database. The `password` and `md5` authentication methods make use of passwords. Database passwords are separate from operating system passwords. Specify a password upon role creation with `CREATE ROLE name PASSWORD 'string'`.

inheritance of privileges

A role inherits the privileges of roles it is a member of, by default. However, to create a role which does not inherit privileges by default, use `CREATE ROLE name NOINHERIT`. Alternatively, inheritance can be overridden for individual grants by using `WITH INHERIT TRUE` or `WITH INHERIT FALSE`.

bypassing row-level security

A role must be explicitly given permission to bypass every row-level security (RLS) policy (except for superusers, since those bypass all permission checks). To create such a role, use `CREATE ROLE name BYPASSRLS` as a superuser.

connection limit

Connection limit can specify how many concurrent connections a role can make. `-1` (the default) means no limit. Specify connection limit upon role creation with `CREATE ROLE name CONNECTION LIMIT 'integer'`.

A role's attributes can be modified after creation with `ALTER ROLE`. See the reference pages for the `CREATE ROLE` and `ALTER ROLE` commands for details.

A role can also have role-specific defaults for many of the run-time configuration settings described in Chapter 19. For example, if for some reason you want to disable index scans (hint: not a good idea) anytime you connect, you can use:

```
ALTER ROLE myname SET enable_indexscan TO off;
```

This will save the setting (but not set it immediately). In subsequent connections by this role it will appear as though `SET enable_indexscan TO off` had been executed just before the session started. You can still alter this setting during the session; it will only be the default. To remove a role-specific default setting, use `ALTER ROLE rolename RESET varname`. Note that role-specific defaults attached to roles without `LOGIN` privilege are fairly useless, since they will never be invoked.

When a non-superuser creates a role using the `CREATEROLE` privilege, the created role is automatically granted back to the creating user, just as if the bootstrap superuser had executed the command `GRANT created_user TO creating_user WITH ADMIN TRUE, SET FALSE, INHERIT FALSE`. Since a `CREATEROLE` user can only exercise special privileges with regard to an existing role if they have `ADMIN OPTION` on it, this grant is just sufficient to allow a `CREATEROLE` user to administer the roles they created. However, because it is created with `INHERIT FALSE, SET FALSE`, the `CREATEROLE` user doesn't inherit the privileges of the created role, nor can it access

the privileges of that role using `SET ROLE`. However, since any user who has `ADMIN OPTION` on a role can grant membership in that role to any other user, the `CREATEROLE` user can gain access to the created role by simply granting that role back to themselves with the `INHERIT` and/or `SET` options. Thus, the fact that privileges are not inherited by default nor is `SET ROLE` granted by default is a safeguard against accidents, not a security feature. Also note that, because this automatic grant is granted by the bootstrap superuser, it cannot be removed or changed by the `CREATEROLE` user; however, any superuser could revoke it, modify it, and/or issue additional such grants to other `CREATEROLE` users. Whichever `CREATEROLE` users have `ADMIN OPTION` on a role at any given time can administer it.

21.3. Role Membership

It is frequently convenient to group users together to ease management of privileges: that way, privileges can be granted to, or revoked from, a group as a whole. In PostgreSQL this is done by creating a role that represents the group, and then granting *membership* in the group role to individual user roles.

To set up a group role, first create the role:

```
CREATE ROLE name;
```

Typically a role being used as a group would not have the `LOGIN` attribute, though you can set it if you wish.

Once the group role exists, you can add and remove members using the `GRANT` and `REVOKE` commands:

```
GRANT group_role TO role1, ... ;  
REVOKE group_role FROM role1, ... ;
```

You can grant membership to other group roles, too (since there isn't really any distinction between group roles and non-group roles). The database will not let you set up circular membership loops. Also, it is not permitted to grant membership in a role to `PUBLIC`.

The members of a group role can use the privileges of the role in two ways. First, member roles that have been granted membership with the `SET` option can do `SET ROLE` to temporarily “become” the group role. In this state, the database session has access to the privileges of the group role rather than the original login role, and any database objects created are considered owned by the group role not the login role. Second, member roles that have been granted membership with the `INHERIT` option automatically have use of the privileges of those directly or indirectly a member of, though the chain stops at memberships lacking the inherit option. As an example, suppose we have done:

```
CREATE ROLE joe LOGIN;  
CREATE ROLE admin;  
CREATE ROLE wheel;  
CREATE ROLE island;  
GRANT admin TO joe WITH INHERIT TRUE;  
GRANT wheel TO admin WITH INHERIT FALSE;  
GRANT island TO joe WITH INHERIT TRUE, SET FALSE;
```

Immediately after connecting as role `joe`, a database session will have use of privileges granted directly to `joe` plus any privileges granted to `admin` and `island`, because `joe` “inherits” those privileges. However, privileges granted to `wheel` are not available, because even though `joe` is indirectly a member of `wheel`, the membership is via `admin` which was granted using `WITH INHERIT FALSE`. After:

```
SET ROLE admin;
```

the session would have use of only those privileges granted to `admin`, and not those granted to `joe` or `island`. After:

```
SET ROLE wheel;
```

the session would have use of only those privileges granted to `wheel`, and not those granted to either `joe` or `admin`. The original privilege state can be restored with any of:

```
SET ROLE joe;  
SET ROLE NONE;  
RESET ROLE;
```

Note

The `SET ROLE` command always allows selecting any role that the original login role is directly or indirectly a member of, provided that there is a chain of membership grants each of which has `SET TRUE` (which is the default). Thus, in the above example, it is not necessary to become `admin` before becoming `wheel`. On the other hand, it is not possible to become `island` at all; `joe` can only access those privileges via inheritance.

Note

In the SQL standard, there is a clear distinction between users and roles, and users do not automatically inherit privileges while roles do. This behavior can be obtained in PostgreSQL by giving roles being used as SQL roles the `INHERIT` attribute, while giving roles being used as SQL users the `NOINHERIT` attribute. However, PostgreSQL defaults to giving all roles the `INHERIT` attribute, for backward compatibility with pre-8.1 releases in which users always had use of permissions granted to groups they were members of.

The role attributes `LOGIN`, `SUPERUSER`, `CREATEDB`, and `CREATEROLE` can be thought of as special privileges, but they are never inherited as ordinary privileges on database objects are. You must actually `SET ROLE` to a specific role having one of these attributes in order to make use of the attribute. Continuing the above example, we might choose to grant `CREATEDB` and `CREATEROLE` to the `admin` role. Then a session connecting as role `joe` would not have these privileges immediately, only after doing `SET ROLE admin`.

To destroy a group role, use `DROP ROLE`:

```
DROP ROLE name;
```

Any memberships in the group role are automatically revoked (but the member roles are not otherwise affected).

21.4. Dropping Roles

Because roles can own database objects and can hold privileges to access other objects, dropping a role is often not just a matter of a quick `DROP ROLE`. Any objects owned by the role must first be dropped or reassigned to other owners; and any permissions granted to the role must be revoked.

Ownership of objects can be transferred one at a time using `ALTER` commands, for example:

```
ALTER TABLE bobs_table OWNER TO alice;
```

Alternatively, the `REASSIGN OWNED` command can be used to reassign ownership of all objects owned by the role-to-be-dropped to a single other role. Because `REASSIGN OWNED` cannot access objects in other databases, it is necessary to run it in each database that contains objects owned by the role. (Note that the first such `REASSIGN OWNED` will change the ownership of any shared-across-databases objects, that is databases or tablespaces, that are owned by the role-to-be-dropped.)

Once any valuable objects have been transferred to new owners, any remaining objects owned by the role-to-be-dropped can be dropped with the `DROP OWNED` command. Again, this command cannot access objects in other databases, so it is necessary to run it in each database that contains objects owned by the role. Also, `DROP OWNED` will not drop entire databases or tablespaces, so it is necessary to do that manually if the role owns any databases or tablespaces that have not been transferred to new owners.

`DROP OWNED` also takes care of removing any privileges granted to the target role for objects that do not belong to it. Because `REASSIGN OWNED` does not touch such objects, it's typically necessary to run both `REASSIGN OWNED` and `DROP OWNED` (in that order!) to fully remove the dependencies of a role to be dropped.

In short then, the most general recipe for removing a role that has been used to own objects is:

```
REASSIGN OWNED BY doomed_role TO successor_role;
DROP OWNED BY doomed_role;
-- repeat the above commands in each database of the cluster
DROP ROLE doomed_role;
```

When not all owned objects are to be transferred to the same successor owner, it's best to handle the exceptions manually and then perform the above steps to mop up.

If `DROP ROLE` is attempted while dependent objects still remain, it will issue messages identifying which objects need to be reassigned or dropped.

21.5. Predefined Roles

PostgreSQL provides a set of predefined roles that provide access to certain, commonly needed, privileged capabilities and information. Administrators (including roles that have the `CREATEROLE` privilege) can `GRANT` these roles to users and/or other roles in their environment, providing those users with access to the specified capabilities and information.

The predefined roles are described in Table 21.1. Note that the specific permissions for each of the roles may change in the future as additional capabilities are added. Administrators should monitor the release notes for changes.

Table 21.1. Predefined Roles

Role	Allowed Access
<code>pg_read_all_data</code>	Read all data (tables, views, sequences), as if having <code>SELECT</code> rights on those objects, and <code>USAGE</code> rights on all schemas, even without having it explicitly. This role does not have the role attribute <code>BYPASSRLS</code> set. If RLS is being used, an administrator may wish to set <code>BYPASSRLS</code> on roles which this role is <code>GRANTED</code> to.
<code>pg_write_all_data</code>	Write all data (tables, views, sequences), as if having <code>INSERT</code> , <code>UPDATE</code> , and <code>DELETE</code> rights on those objects, and <code>USAGE</code> rights on all schemas, even without having it explicitly. This role

Role	Allowed Access
	does not have the role attribute <code>BYPASSRLS</code> set. If RLS is being used, an administrator may wish to set <code>BYPASSRLS</code> on roles which this role is GRANTED to.
<code>pg_read_all_settings</code>	Read all configuration variables, even those normally visible only to superusers.
<code>pg_read_all_stats</code>	Read all <code>pg_stat_*</code> views and use various statistics related extensions, even those normally visible only to superusers.
<code>pg_stat_scan_tables</code>	Execute monitoring functions that may take <code>ACCESS SHARE</code> locks on tables, potentially for a long time.
<code>pg_monitor</code>	Read/execute various monitoring views and functions. This role is a member of <code>pg_read_all_settings</code> , <code>pg_read_all_stats</code> and <code>pg_stat_scan_tables</code> .
<code>pg_database_owner</code>	None. Membership consists, implicitly, of the current database owner.
<code>pg_signal_backend</code>	Signal another backend to cancel a query or terminate its session.
<code>pg_read_server_files</code>	Allow reading files from any location the database can access on the server with <code>COPY</code> and other file-access functions.
<code>pg_write_server_files</code>	Allow writing to files in any location the database can access on the server with <code>COPY</code> and other file-access functions.
<code>pg_execute_server_program</code>	Allow executing programs on the database server as the user the database runs as with <code>COPY</code> and other functions which allow executing a server-side program.
<code>pg_checkpoint</code>	Allow executing the <code>CHECKPOINT</code> command.
<code>pg_maintain</code>	Allow executing <code>VACUUM</code> , <code>ANALYZE</code> , <code>CLUSTER</code> , <code>REFRESH MATERIALIZED VIEW</code> , <code>REINDEX</code> , and <code>LOCK TABLE</code> on all relations, as if having <code>MAINTAIN</code> rights on those objects, even without having it explicitly.
<code>pg_use_reserved_connections</code>	Allow use of connection slots reserved via <code>reserved_connections</code> .
<code>pg_create_subscription</code>	Allow users with <code>CREATE</code> permission on the database to issue <code>CREATE SUBSCRIPTION</code> .

The `pg_monitor`, `pg_read_all_settings`, `pg_read_all_stats` and `pg_stat_scan_tables` roles are intended to allow administrators to easily configure a role for the purpose of monitoring the database server. They grant a set of common privileges allowing the role to read various useful configuration settings, statistics and other system information normally restricted to superusers.

The `pg_database_owner` role has one implicit, situation-dependent member, namely the owner of the current database. Like any role, it can own objects or receive grants of access privileges. Consequently, once `pg_database_owner` has rights within a template database, each owner of a database instantiated from that template will exercise those rights. `pg_database_owner` cannot be a member of any role, and it cannot have non-implicit members. Initially, this role owns the `public` schema, so each database owner governs local use of the schema.

The `pg_signal_backend` role is intended to allow administrators to enable trusted, but non-superuser, roles to send signals to other backends. Currently this role enables sending of signals for canceling a query on another backend or terminating its session. A user granted this role cannot however send signals to a backend owned by a superuser. See Section 9.28.2.

The `pg_read_server_files`, `pg_write_server_files` and `pg_execute_server_program` roles are intended to allow administrators to have trusted, but non-superuser, roles which are able to access files and run programs on the database server as the user the database runs as. As these roles are able to access any file on the server file system, they bypass all database-level

permission checks when accessing files directly and they could be used to gain superuser-level access, therefore great care should be taken when granting these roles to users.

Care should be taken when granting these roles to ensure they are only used where needed and with the understanding that these roles grant access to privileged information.

Administrators can grant access to these roles to users using the **GRANT** command, for example:

```
GRANT pg_signal_backend TO admin_user;
```

21.6. Function Security

Functions, triggers and row-level security policies allow users to insert code into the backend server that other users might execute unintentionally. Hence, these mechanisms permit users to “Trojan horse” others with relative ease. The strongest protection is tight control over who can define objects. Where that is infeasible, write queries referring only to objects having trusted owners. Remove from `search_path` any schemas that permit untrusted users to create objects.

Functions run inside the backend server process with the operating system permissions of the database server daemon. If the programming language used for the function allows unchecked memory accesses, it is possible to change the server's internal data structures. Hence, among many other things, such functions can circumvent any system access controls. Function languages that allow such access are considered “untrusted”, and PostgreSQL allows only superusers to create functions written in those languages.

Chapter 24. Routine Database Maintenance Tasks

PostgreSQL, like any database software, requires that certain tasks be performed regularly to achieve optimum performance. The tasks discussed here are *required*, but they are repetitive in nature and can easily be automated using standard tools such as cron scripts or Windows' Task Scheduler. It is the database administrator's responsibility to set up appropriate scripts, and to check that they execute successfully.

One obvious maintenance task is the creation of backup copies of the data on a regular schedule. Without a recent backup, you have no chance of recovery after a catastrophe (disk failure, fire, mistakenly dropping a critical table, etc.). The backup and recovery mechanisms available in PostgreSQL are discussed at length in Chapter 25.

The other main category of maintenance task is periodic “vacuuming” of the database. This activity is discussed in Section 24.1. Closely related to this is updating the statistics that will be used by the query planner, as discussed in Section 24.1.3.

Another task that might need periodic attention is log file management. This is discussed in Section 24.3.

`check_postgres`¹ is available for monitoring database health and reporting unusual conditions. `check_postgres` integrates with Nagios and MRTG, but can be run standalone too.

PostgreSQL is low-maintenance compared to some other database management systems. Nonetheless, appropriate attention to these tasks will go far towards ensuring a pleasant and productive experience with the system.

24.1. Routine Vacuuming

PostgreSQL databases require periodic maintenance known as *vacuuming*. For many installations, it is sufficient to let vacuuming be performed by the *autovacuum daemon*, which is described in Section 24.1.6. You might need to adjust the autovacuuming parameters described there to obtain best results for your situation. Some database administrators will want to supplement or replace the daemon's activities with manually-managed `VACUUM` commands, which typically are executed according to a schedule by cron or Task Scheduler scripts. To set up manually-managed vacuuming properly, it is essential to understand the issues discussed in the next few subsections. Administrators who rely on autovacuuming may still wish to skim this material to help them understand and adjust autovacuuming.

24.1.1. Vacuuming Basics

PostgreSQL's `VACUUM` command has to process each table on a regular basis for several reasons:

1. To recover or reuse disk space occupied by updated or deleted rows.
2. To update data statistics used by the PostgreSQL query planner.
3. To update the visibility map, which speeds up index-only scans.
4. To protect against loss of very old data due to *transaction ID wraparound* or *multixact ID wrap-around*.

Each of these reasons dictates performing `VACUUM` operations of varying frequency and scope, as explained in the following subsections.

¹ https://bucardo.org/check_postgres/

There are two variants of `VACUUM`: standard `VACUUM` and `VACUUM FULL`. `VACUUM FULL` can reclaim more disk space but runs much more slowly. Also, the standard form of `VACUUM` can run in parallel with production database operations. (Commands such as `SELECT`, `INSERT`, `UPDATE`, and `DELETE` will continue to function normally, though you will not be able to modify the definition of a table with commands such as `ALTER TABLE` while it is being vacuumed.) `VACUUM FULL` requires an `ACCESS EXCLUSIVE` lock on the table it is working on, and therefore cannot be done in parallel with other use of the table. Generally, therefore, administrators should strive to use standard `VACUUM` and avoid `VACUUM FULL`.

`VACUUM` creates a substantial amount of I/O traffic, which can cause poor performance for other active sessions. There are configuration parameters that can be adjusted to reduce the performance impact of background vacuuming — see Section 19.4.4.

24.1.2. Recovering Disk Space

In PostgreSQL, an `UPDATE` or `DELETE` of a row does not immediately remove the old version of the row. This approach is necessary to gain the benefits of multiversion concurrency control (MVCC, see Chapter 13): the row version must not be deleted while it is still potentially visible to other transactions. But eventually, an outdated or deleted row version is no longer of interest to any transaction. The space it occupies must then be reclaimed for reuse by new rows, to avoid unbounded growth of disk space requirements. This is done by running `VACUUM`.

The standard form of `VACUUM` removes dead row versions in tables and indexes and marks the space available for future reuse. However, it will not return the space to the operating system, except in the special case where one or more pages at the end of a table become entirely free and an exclusive table lock can be easily obtained. In contrast, `VACUUM FULL` actively compacts tables by writing a complete new version of the table file with no dead space. This minimizes the size of the table, but can take a long time. It also requires extra disk space for the new copy of the table, until the operation completes.

The usual goal of routine vacuuming is to do standard `VACUUM`s often enough to avoid needing `VACUUM FULL`. The autovacuum daemon attempts to work this way, and in fact will never issue `VACUUM FULL`. In this approach, the idea is not to keep tables at their minimum size, but to maintain steady-state usage of disk space: each table occupies space equivalent to its minimum size plus however much space gets used up between vacuum runs. Although `VACUUM FULL` can be used to shrink a table back to its minimum size and return the disk space to the operating system, there is not much point in this if the table will just grow again in the future. Thus, moderately-frequent standard `VACUUM` runs are a better approach than infrequent `VACUUM FULL` runs for maintaining heavily-updated tables.

Some administrators prefer to schedule vacuuming themselves, for example doing all the work at night when load is low. The difficulty with doing vacuuming according to a fixed schedule is that if a table has an unexpected spike in update activity, it may get bloated to the point that `VACUUM FULL` is really necessary to reclaim space. Using the autovacuum daemon alleviates this problem, since the daemon schedules vacuuming dynamically in response to update activity. It is unwise to disable the daemon completely unless you have an extremely predictable workload. One possible compromise is to set the daemon's parameters so that it will only react to unusually heavy update activity, thus keeping things from getting out of hand, while scheduled `VACUUM`s are expected to do the bulk of the work when the load is typical.

For those not using autovacuum, a typical approach is to schedule a database-wide `VACUUM` once a day during a low-usage period, supplemented by more frequent vacuuming of heavily-updated tables as necessary. (Some installations with extremely high update rates vacuum their busiest tables as often as once every few minutes.) If you have multiple databases in a cluster, don't forget to `VACUUM` each one; the program `vacuumdb` might be helpful.

Tip

Plain `VACUUM` may not be satisfactory when a table contains large numbers of dead row versions as a result of massive update or delete activity. If you have such a table and you need to

reclaim the excess disk space it occupies, you will need to use `VACUUM FULL`, or alternatively `CLUSTER` or one of the table-rewriting variants of `ALTER TABLE`. These commands rewrite an entire new copy of the table and build new indexes for it. All these options require an `ACCESS EXCLUSIVE` lock. Note that they also temporarily use extra disk space approximately equal to the size of the table, since the old copies of the table and indexes can't be released until the new ones are complete.

Tip

If you have a table whose entire contents are deleted on a periodic basis, consider doing it with `TRUNCATE` rather than using `DELETE` followed by `VACUUM`. `TRUNCATE` removes the entire content of the table immediately, without requiring a subsequent `VACUUM` or `VACUUM FULL` to reclaim the now-unused disk space. The disadvantage is that strict MVCC semantics are violated.

24.1.3. Updating Planner Statistics

The PostgreSQL query planner relies on statistical information about the contents of tables in order to generate good plans for queries. These statistics are gathered by the `ANALYZE` command, which can be invoked by itself or as an optional step in `VACUUM`. It is important to have reasonably accurate statistics, otherwise poor choices of plans might degrade database performance.

The autovacuum daemon, if enabled, will automatically issue `ANALYZE` commands whenever the content of a table has changed sufficiently. However, administrators might prefer to rely on manually-scheduled `ANALYZE` operations, particularly if it is known that update activity on a table will not affect the statistics of “interesting” columns. The daemon schedules `ANALYZE` strictly as a function of the number of rows inserted or updated; it has no knowledge of whether that will lead to meaningful statistical changes.

Tuples changed in partitions and inheritance children do not trigger analyze on the parent table. If the parent table is empty or rarely changed, it may never be processed by autovacuum, and the statistics for the inheritance tree as a whole won't be collected. It is necessary to run `ANALYZE` on the parent table manually in order to keep the statistics up to date.

As with vacuuming for space recovery, frequent updates of statistics are more useful for heavily-updated tables than for seldom-updated ones. But even for a heavily-updated table, there might be no need for statistics updates if the statistical distribution of the data is not changing much. A simple rule of thumb is to think about how much the minimum and maximum values of the columns in the table change. For example, a `timestamp` column that contains the time of row update will have a constantly-increasing maximum value as rows are added and updated; such a column will probably need more frequent statistics updates than, say, a column containing URLs for pages accessed on a website. The URL column might receive changes just as often, but the statistical distribution of its values probably changes relatively slowly.

It is possible to run `ANALYZE` on specific tables and even just specific columns of a table, so the flexibility exists to update some statistics more frequently than others if your application requires it. In practice, however, it is usually best to just analyze the entire database, because it is a fast operation. `ANALYZE` uses a statistically random sampling of the rows of a table rather than reading every single row.

Tip

Although per-column tweaking of `ANALYZE` frequency might not be very productive, you might find it worthwhile to do per-column adjustment of the level of detail of the statistics collected by `ANALYZE`. Columns that are heavily used in `WHERE` clauses and have highly

irregular data distributions might require a finer-grain data histogram than other columns. See `ALTER TABLE SET STATISTICS`, or change the database-wide default using the `default_statistics_target` configuration parameter.

Also, by default there is limited information available about the selectivity of functions. However, if you create a statistics object or an expression index that uses a function call, useful statistics will be gathered about the function, which can greatly improve query plans that use the expression index.

Tip

The autovacuum daemon does not issue `ANALYZE` commands for foreign tables, since it has no means of determining how often that might be useful. If your queries require statistics on foreign tables for proper planning, it's a good idea to run manually-managed `ANALYZE` commands on those tables on a suitable schedule.

Tip

The autovacuum daemon does not issue `ANALYZE` commands for partitioned tables. Inheritance parents will only be analyzed if the parent itself is changed - changes to child tables do not trigger autoanalyze on the parent table. If your queries require statistics on parent tables for proper planning, it is necessary to periodically run a manual `ANALYZE` on those tables to keep the statistics up to date.

24.1.4. Updating the Visibility Map

Vacuum maintains a visibility map for each table to keep track of which pages contain only tuples that are known to be visible to all active transactions (and all future transactions, until the page is again modified). This has two purposes. First, vacuum itself can skip such pages on the next run, since there is nothing to clean up.

Second, it allows PostgreSQL to answer some queries using only the index, without reference to the underlying table. Since PostgreSQL indexes don't contain tuple visibility information, a normal index scan fetches the heap tuple for each matching index entry, to check whether it should be seen by the current transaction. An *index-only scan*, on the other hand, checks the visibility map first. If it's known that all tuples on the page are visible, the heap fetch can be skipped. This is most useful on large data sets where the visibility map can prevent disk accesses. The visibility map is vastly smaller than the heap, so it can easily be cached even when the heap is very large.

24.1.5. Preventing Transaction ID Wraparound Failures

PostgreSQL's MVCC transaction semantics depend on being able to compare transaction ID (XID) numbers: a row version with an insertion XID greater than the current transaction's XID is “in the future” and should not be visible to the current transaction. But since transaction IDs have limited size (32 bits) a cluster that runs for a long time (more than 4 billion transactions) would suffer *transaction ID wraparound*: the XID counter wraps around to zero, and all of a sudden transactions that were in the past appear to be in the future — which means their output become invisible. In short, catastrophic data loss. (Actually the data is still there, but that's cold comfort if you cannot get at it.) To avoid this, it is necessary to vacuum every table in every database at least once every two billion transactions.

The reason that periodic vacuuming solves the problem is that `VACUUM` will mark rows as *frozen*, indicating that they were inserted by a transaction that committed sufficiently far in the past that the effects of the inserting transaction are certain to be visible to all current and future transactions.

Normal XIDs are compared using modulo-2³² arithmetic. This means that for every normal XID, there are two billion XIDs that are “older” and two billion that are “newer”; another way to say it is that the normal XID space is circular with no endpoint. Therefore, once a row version has been created with a particular normal XID, the row version will appear to be “in the past” for the next two billion transactions, no matter which normal XID we are talking about. If the row version still exists after more than two billion transactions, it will suddenly appear to be in the future. To prevent this, PostgreSQL reserves a special XID, `FrozenTransactionId`, which does not follow the normal XID comparison rules and is always considered older than every normal XID. Frozen row versions are treated as if the inserting XID were `FrozenTransactionId`, so that they will appear to be “in the past” to all normal transactions regardless of wraparound issues, and so such row versions will be valid until deleted, no matter how long that is.

Note

In PostgreSQL versions before 9.4, freezing was implemented by actually replacing a row's insertion XID with `FrozenTransactionId`, which was visible in the row's `xmin` system column. Newer versions just set a flag bit, preserving the row's original `xmin` for possible forensic use. However, rows with `xmin` equal to `FrozenTransactionId` (2) may still be found in databases pg_upgrade'd from pre-9.4 versions.

Also, system catalogs may contain rows with `xmin` equal to `BootstrapTransactionId` (1), indicating that they were inserted during the first phase of `initdb`. Like `FrozenTransactionId`, this special XID is treated as older than every normal XID.

`vacuum_freeze_min_age` controls how old an XID value has to be before rows bearing that XID will be frozen. Increasing this setting may avoid unnecessary work if the rows that would otherwise be frozen will soon be modified again, but decreasing this setting increases the number of transactions that can elapse before the table must be vacuumed again.

`VACUUM` uses the visibility map to determine which pages of a table must be scanned. Normally, it will skip pages that don't have any dead row versions even if those pages might still have row versions with old XID values. Therefore, normal `VACUUM`s won't always freeze every old row version in the table. When that happens, `VACUUM` will eventually need to perform an *aggressive vacuum*, which will freeze all eligible unfrozen XID and MXID values, including those from all-visible but not all-frozen pages. In practice most tables require periodic aggressive vacuuming. `vacuum_freeze_table_age` controls when `VACUUM` does that: all-visible but not all-frozen pages are scanned if the number of transactions that have passed since the last such scan is greater than `vacuum_freeze_table_age` minus `vacuum_freeze_min_age`. Setting `vacuum_freeze_table_age` to 0 forces `VACUUM` to always use its aggressive strategy.

The maximum time that a table can go unvacuumed is two billion transactions minus the `vacuum_freeze_min_age` value at the time of the last aggressive vacuum. If it were to go unvacuumed for longer than that, data loss could result. To ensure that this does not happen, autovacuum is invoked on any table that might contain unfrozen rows with XIDs older than the age specified by the configuration parameter `autovacuum_freeze_max_age`. (This will happen even if autovacuum is disabled.)

This implies that if a table is not otherwise vacuumed, autovacuum will be invoked on it approximately once every `autovacuum_freeze_max_age` minus `vacuum_freeze_min_age` transactions. For tables that are regularly vacuumed for space reclamation purposes, this is of little importance. However, for static tables (including tables that receive inserts, but no updates or deletes), there is no need to vacuum for space reclamation, so it can be useful to try to maximize the interval between forced autovacua on very large static tables. Obviously one can do this either by increasing `autovacuum_freeze_max_age` or decreasing `vacuum_freeze_min_age`.

The effective maximum for `vacuum_freeze_table_age` is `0.95 * autovacuum_freeze_max_age`; a setting higher than that will be capped to the maximum. A value higher than `autovacuum_freeze_max_age` wouldn't make sense because an anti-wraparound autovac-

uum would be triggered at that point anyway, and the 0.95 multiplier leaves some breathing room to run a manual `VACUUM` before that happens. As a rule of thumb, `vacuum_freeze_table_age` should be set to a value somewhat below `autovacuum_freeze_max_age`, leaving enough gap so that a regularly scheduled `VACUUM` or an autovacuum triggered by normal delete and update activity is run in that window. Setting it too close could lead to anti-wraparound autovacua, even though the table was recently vacuumed to reclaim space, whereas lower values lead to more frequent aggressive vacuuming.

The sole disadvantage of increasing `autovacuum_freeze_max_age` (and `vacuum_freeze_table_age` along with it) is that the `pg_xact` and `pg_commit_ts` subdirectories of the database cluster will take more space, because it must store the commit status (if `track_commit_timestamp` is enabled) timestamp of all transactions back to the `autovacuum_freeze_max_age` horizon. The commit status uses two bits per transaction, so if `autovacuum_freeze_max_age` is set to its maximum allowed value of two billion, `pg_xact` can be expected to grow to about half a gigabyte and `pg_commit_ts` to about 20GB. If this is trivial compared to your total database size, setting `autovacuum_freeze_max_age` to its maximum allowed value is recommended. Otherwise, set it depending on what you are willing to allow for `pg_xact` and `pg_commit_ts` storage. (The default, 200 million transactions, translates to about 50MB of `pg_xact` storage and about 2GB of `pg_commit_ts` storage.)

One disadvantage of decreasing `vacuum_freeze_min_age` is that it might cause `VACUUM` to do useless work: freezing a row version is a waste of time if the row is modified soon thereafter (causing it to acquire a new `XID`). So the setting should be large enough that rows are not frozen until they are unlikely to change any more.

To track the age of the oldest unfrozen `XIDs` in a database, `VACUUM` stores `XID` statistics in the system tables `pg_class` and `pg_database`. In particular, the `relfrozenxid` column of a table's `pg_class` row contains the oldest remaining unfrozen `XID` at the end of the most recent `VACUUM` that successfully advanced `relfrozenxid` (typically the most recent aggressive `VACUUM`). Similarly, the `datfrozenxid` column of a database's `pg_database` row is a lower bound on the unfrozen `XIDs` appearing in that database — it is just the minimum of the per-table `relfrozenxid` values within the database. A convenient way to examine this information is to execute queries such as:

```
SELECT c.oid::regclass as table_name,  
       greatest(age(c.relfrozenxid),age(t.relfrozenxid)) as age  
FROM pg_class c  
LEFT JOIN pg_class t ON c.reltoastrelid = t.oid  
WHERE c.relkind IN ('r', 'm');
```

```
SELECT datname, age(datfrozenxid) FROM pg_database;
```

The `age` column measures the number of transactions from the cutoff `XID` to the current transaction's `XID`.

Tip

When the `VACUUM` command's `VERBOSE` parameter is specified, `VACUUM` prints various statistics about the table. This includes information about how `relfrozenxid` and `relminmxid` advanced, and the number of newly frozen pages. The same details appear in the server log when autovacuum logging (controlled by `log_autovacuum_min_duration`) reports on a `VACUUM` operation executed by autovacuum.

`VACUUM` normally only scans pages that have been modified since the last vacuum, but `relfrozenxid` can only be advanced when every page of the table that might contain unfrozen `XIDs` is scanned. This happens when `relfrozenxid` is more than `vacuum_freeze_table_age` transactions old, when `VACUUM`'s `FREEZE` option is used, or when all pages that are not already all-frozen

happen to require vacuuming to remove dead row versions. When `VACUUM` scans every page in the table that is not already all-frozen, it should set `age(relfrozenxid)` to a value just a little more than the `vacuum_freeze_min_age` setting that was used (more by the number of transactions started since the `VACUUM` started). `VACUUM` will set `relfrozenxid` to the oldest `XID` that remains in the table, so it's possible that the final value will be much more recent than strictly required. If no `relfrozenxid`-advancing `VACUUM` is issued on the table until `autovacuum_freeze_max_age` is reached, an autovacuum will soon be forced for the table.

If for some reason autovacuum fails to clear old `XIDs` from a table, the system will begin to emit warning messages like this when the database's oldest `XIDs` reach forty million transactions from the wraparound point:

```
WARNING: database "mydb" must be vacuumed within 39985967
transactions
HINT: To avoid XID assignment failures, execute a database-wide
VACUUM in that database.
```

(A manual `VACUUM` should fix the problem, as suggested by the hint; but note that the `VACUUM` should be performed by a superuser, else it will fail to process system catalogs, which prevent it from being able to advance the database's `datfrozenxid`.) If these warnings are ignored, the system will refuse to assign new `XIDs` once there are fewer than three million transactions left until wraparound:

```
ERROR: database is not accepting commands that assign new XIDs to
avoid wraparound data loss in database "mydb"
HINT: Execute a database-wide VACUUM in that database.
```

In this condition any transactions already in progress can continue, but only read-only transactions can be started. Operations that modify database records or truncate relations will fail. The `VACUUM` command can still be run normally. Note that, contrary to what was sometimes recommended in earlier releases, it is not necessary or desirable to stop the postmaster or enter single user-mode in order to restore normal operation. Instead, follow these steps:

1. Resolve old prepared transactions. You can find these by checking `pg_prepared_xacts` for rows where `age(transactionid)` is large. Such transactions should be committed or rolled back.
2. End long-running open transactions. You can find these by checking `pg_stat_activity` for rows where `age(backend_xid)` or `age(backend_xmin)` is large. Such transactions should be committed or rolled back, or the session can be terminated using `pg_terminate_backend`.
3. Drop any old replication slots. Use `pg_stat_replication` to find slots where `age(xmin)` or `age/catalog_xmin)` is large. In many cases, such slots were created for replication to servers that no longer exist, or that have been down for a long time. If you drop a slot for a server that still exists and might still try to connect to that slot, that replica may need to be rebuilt.
4. Execute `VACUUM` in the target database. A database-wide `VACUUM` is simplest; to reduce the time required, it is also possible to issue manual `VACUUM` commands on the tables where `relminxid` is oldest. Do not use `VACUUM FULL` in this scenario, because it requires an `XID` and will therefore fail, except in super-user mode, where it will instead consume an `XID` and thus increase the risk of transaction ID wraparound. Do not use `VACUUM FREEZE` either, because it will do more than the minimum amount of work required to restore normal operation.
5. Once normal operation is restored, ensure that autovacuum is properly configured in the target database in order to avoid future problems.

Note

In earlier versions, it was sometimes necessary to stop the postmaster and `VACUUM` the database in a single-user mode. In typical scenarios, this is no longer necessary, and should be

avoided whenever possible, since it involves taking the system down. It is also riskier, since it disables transaction ID wraparound safeguards that are designed to prevent data loss. The only reason to use single-user mode in this scenario is if you wish to **TRUNCATE** or **DROP** unneeded tables to avoid needing to **VACUUM** them. The three-million-transaction safety margin exists to let the administrator do this. See the postgres reference page for details about using single-user mode.

24.1.5.1. Multixacts and Wraparound

Multixact IDs are used to support row locking by multiple transactions. Since there is only limited space in a tuple header to store lock information, that information is encoded as a “multiple transaction ID”, or multixact ID for short, whenever there is more than one transaction concurrently locking a row. Information about which transaction IDs are included in any particular multixact ID is stored separately in the `pg_multixact` subdirectory, and only the multixact ID appears in the `xmax` field in the tuple header. Like transaction IDs, multixact IDs are implemented as a 32-bit counter and corresponding storage, all of which requires careful aging management, storage cleanup, and wraparound handling. There is a separate storage area which holds the list of members in each multixact, which also uses a 32-bit counter and which must also be managed. The system function `pg_get_multixact_members()` described in Table 9.82 can be used to examine the transaction IDs associated with a multixact ID.

Whenever **VACUUM** scans any part of a table, it will replace any multixact ID it encounters which is older than `vacuum_multixact_freeze_min_age` by a different value, which can be the zero value, a single transaction ID, or a newer multixact ID. For each table, `pg_class.relmixmapid` stores the oldest possible multixact ID still appearing in any tuple of that table. If this value is older than `vacuum_multixact_freeze_table_age`, an aggressive vacuum is forced. As discussed in the previous section, an aggressive vacuum means that only those pages which are known to be all-frozen will be skipped. `mxid_age()` can be used on `pg_class.relmixmapid` to find its age.

Aggressive **VACUUMs**, regardless of what causes them, are *guaranteed* to be able to advance the table's `relmixmapid`. Eventually, as all tables in all databases are scanned and their oldest multixact values are advanced, on-disk storage for older multixacts can be removed.

As a safety device, an aggressive vacuum scan will occur for any table whose multixact-age is greater than `autovacuum_multixact_freeze_max_age`. Also, if the storage occupied by multixacts members exceeds about 10GB, aggressive vacuum scans will occur more often for all tables, starting with those that have the oldest multixact-age. Both of these kinds of aggressive scans will occur even if autovacuum is nominally disabled. The members storage area can grow up to about 20GB before reaching wraparound.

Similar to the **XID** case, if autovacuum fails to clear old **MXIDs** from a table, the system will begin to emit warning messages when the database's oldest **MXIDs** reach forty million transactions from the wraparound point. And, just as in the **XID** case, if these warnings are ignored, the system will refuse to generate new **MXIDs** once there are fewer than three million left until wraparound.

Normal operation when **MXIDs** are exhausted can be restored in much the same way as when **XIDs** are exhausted. Follow the same steps in the previous section, but with the following differences:

1. Running transactions and prepared transactions can be ignored if there is no chance that they might appear in a multixact.
2. **MXID** information is not directly visible in system views such as `pg_stat_activity`; however, looking for old **XIDs** is still a good way of determining which transactions are causing **MXID** wraparound problems.
3. **XID** exhaustion will block all write transactions, but **MXID** exhaustion will only block a subset of write transactions, specifically those that involve row locks that require an **MXID**.

24.1.6. The Autovacuum Daemon

PostgreSQL has an optional but highly recommended feature called *autovacuum*, whose purpose is to automate the execution of `VACUUM` and `ANALYZE` commands. When enabled, autovacuum checks for tables that have had a large number of inserted, updated or deleted tuples. These checks use the statistics collection facility; therefore, autovacuum cannot be used unless `track_counts` is set to `true`. In the default configuration, autovacuuming is enabled and the related configuration parameters are appropriately set.

The “autovacuum daemon” actually consists of multiple processes. There is a persistent daemon process, called the *autovacuum launcher*, which is in charge of starting *autovacuum worker* processes for all databases. The launcher will distribute the work across time, attempting to start one worker within each database every `autovacuum_naptime` seconds. (Therefore, if the installation has *N* databases, a new worker will be launched every `autovacuum_naptime/N` seconds.) A maximum of `autovacuum_max_workers` worker processes are allowed to run at the same time. If there are more than `autovacuum_max_workers` databases to be processed, the next database will be processed as soon as the first worker finishes. Each worker process will check each table within its database and execute `VACUUM` and/or `ANALYZE` as needed. `log_autovacuum_min_duration` can be set to monitor autovacuum workers' activity.

If several large tables all become eligible for vacuuming in a short amount of time, all autovacuum workers might become occupied with vacuuming those tables for a long period. This would result in other tables and databases not being vacuumed until a worker becomes available. There is no limit on how many workers might be in a single database, but workers do try to avoid repeating work that has already been done by other workers. Note that the number of running workers does not count towards `max_connections` or `superuser_reserved_connections` limits.

Tables whose `relfrozenxid` value is more than `autovacuum_freeze_max_age` transactions old are always vacuumed (this also applies to those tables whose freeze max age has been modified via storage parameters; see below). Otherwise, if the number of tuples obsoleted since the last `VACUUM` exceeds the “vacuum threshold”, the table is vacuumed. The vacuum threshold is defined as:

$$\text{vacuum threshold} = \text{vacuum base threshold} + \text{vacuum scale factor} * \text{number of tuples}$$

where the vacuum base threshold is `autovacuum_vacuum_threshold`, the vacuum scale factor is `autovacuum_vacuum_scale_factor`, and the number of tuples is `pg_class.rel tuples`.

The table is also vacuumed if the number of tuples inserted since the last vacuum has exceeded the defined insert threshold, which is defined as:

$$\text{vacuum insert threshold} = \text{vacuum base insert threshold} + \text{vacuum insert scale factor} * \text{number of tuples}$$

where the vacuum insert base threshold is `autovacuum_vacuum_insert_threshold`, and vacuum insert scale factor is `autovacuum_vacuum_insert_scale_factor`. Such vacuums may allow portions of the table to be marked as *all visible* and also allow tuples to be frozen, which can reduce the work required in subsequent vacuums. For tables which receive `INSERT` operations but no or almost no `UPDATE/DELETE` operations, it may be beneficial to lower the table's `autovacuum_freeze_min_age` as this may allow tuples to be frozen by earlier vacuums. The number of obsolete tuples and the number of inserted tuples are obtained from the cumulative statistics system; it is an eventually-consistent count updated by each `UPDATE`, `DELETE` and `INSERT` operation. If the `relfrozenxid` value of the table is more than `vacuum_freeze_table_age` transactions old, an aggressive vacuum is performed to freeze old tuples and advance `relfrozenxid`; otherwise, only pages that have been modified since the last vacuum are scanned.

For analyze, a similar condition is used: the threshold, defined as:

`analyze threshold = analyze base threshold + analyze scale factor *
number of tuples`

is compared to the total number of tuples inserted, updated, or deleted since the last `ANALYZE`.

Partitioned tables do not directly store tuples and consequently are not processed by autovacuum. (Autovacuum does process table partitions just like other tables.) Unfortunately, this means that autovacuum does not run `ANALYZE` on partitioned tables, and this can cause suboptimal plans for queries that reference partitioned table statistics. You can work around this problem by manually running `ANALYZE` on partitioned tables when they are first populated, and again whenever the distribution of data in their partitions changes significantly.

Temporary tables cannot be accessed by autovacuum. Therefore, appropriate vacuum and analyze operations should be performed via session SQL commands.

The default thresholds and scale factors are taken from `postgresql.conf`, but it is possible to override them (and many other autovacuum control parameters) on a per-table basis; see Storage Parameters for more information. If a setting has been changed via a table's storage parameters, that value is used when processing that table; otherwise the global settings are used. See Section 19.10 for more details on the global settings.

When multiple workers are running, the autovacuum cost delay parameters (see Section 19.4.4) are “balanced” among all the running workers, so that the total I/O impact on the system is the same regardless of the number of workers actually running. However, any workers processing tables whose per-table `autovacuum_vacuum_cost_delay` or `autovacuum_vacuum_cost_limit` storage parameters have been set are not considered in the balancing algorithm.

Autovacuum workers generally don't block other commands. If a process attempts to acquire a lock that conflicts with the `SHARE UPDATE EXCLUSIVE` lock held by autovacuum, lock acquisition will interrupt the autovacuum. For conflicting lock modes, see Table 13.2. However, if the autovacuum is running to prevent transaction ID wraparound (i.e., the autovacuum query name in the `pg_stat_activity` view ends with `(to prevent wraparound)`), the autovacuum is not automatically interrupted.

Warning

Regularly running commands that acquire locks conflicting with a `SHARE UPDATE EXCLUSIVE` lock (e.g., `ANALYZE`) can effectively prevent autovacuum from ever completing.

24.2. Routine Reindexing

In some situations it is worthwhile to rebuild indexes periodically with the `REINDEX` command or a series of individual rebuilding steps.

B-tree index pages that have become completely empty are reclaimed for re-use. However, there is still a possibility of inefficient use of space: if all but a few index keys on a page have been deleted, the page remains allocated. Therefore, a usage pattern in which most, but not all, keys in each range are eventually deleted will see poor use of space. For such usage patterns, periodic reindexing is recommended.

The potential for bloat in non-B-tree indexes has not been well researched. It is a good idea to periodically monitor the index's physical size when using any non-B-tree index type.

Also, for B-tree indexes, a freshly-constructed index is slightly faster to access than one that has been updated many times because logically adjacent pages are usually also physically adjacent in a newly built index. (This consideration does not apply to non-B-tree indexes.) It might be worthwhile to reindex periodically just to improve access speed.

REINDEX can be used safely and easily in all cases. This command requires an ACCESS EXCLUSIVE lock by default, hence it is often preferable to execute it with its CONCURRENTLY option, which requires only a SHARE UPDATE EXCLUSIVE lock.

24.3. Log File Maintenance

It is a good idea to save the database server's log output somewhere, rather than just discarding it via `/dev/null`. The log output is invaluable when diagnosing problems.

Note

The server log can contain sensitive information and needs to be protected, no matter how or where it is stored, or the destination to which it is routed. For example, some DDL statements might contain plaintext passwords or other authentication details. Logged statements at the ERROR level might show the SQL source code for applications and might also contain some parts of data rows. Recording data, events and related information is the intended function of this facility, so this is not a leakage or a bug. Please ensure the server logs are visible only to appropriately authorized people.

Log output tends to be voluminous (especially at higher debug levels) so you won't want to save it indefinitely. You need to *rotate* the log files so that new log files are started and old ones removed after a reasonable period of time.

If you simply direct the stderr of `postgres` into a file, you will have log output, but the only way to truncate the log file is to stop and restart the server. This might be acceptable if you are using PostgreSQL in a development environment, but few production servers would find this behavior acceptable.

A better approach is to send the server's stderr output to some type of log rotation program. There is a built-in log rotation facility, which you can use by setting the configuration parameter `logging_collector` to `true` in `postgresql.conf`. The control parameters for this program are described in Section 19.8.1. You can also use this approach to capture the log data in machine readable CSV (comma-separated values) format.

Alternatively, you might prefer to use an external log rotation program if you have one that you are already using with other server software. For example, the `rotatelogs` tool included in the Apache distribution can be used with PostgreSQL. One way to do this is to pipe the server's stderr output to the desired program. If you start the server with `pg_ctl`, then stderr is already redirected to stdout, so you just need a pipe command, for example:

```
pg_ctl start | rotatelogs /var/log/pgsql_log 86400
```

You can combine these approaches by setting up `logrotate` to collect log files produced by PostgreSQL built-in logging collector. In this case, the logging collector defines the names and location of the log files, while `logrotate` periodically archives these files. When initiating log rotation, `logrotate` must ensure that the application sends further output to the new file. This is commonly done with a `postrotate` script that sends a `SIGHUP` signal to the application, which then reopens the log file. In PostgreSQL, you can run `pg_ctl` with the `logrotate` option instead. When the server receives this command, the server either switches to a new log file or reopens the existing file, depending on the logging configuration (see Section 19.8.1).

Note

When using static log file names, the server might fail to reopen the log file if the max open file limit is reached or a file table overflow occurs. In this case, log messages are sent to the old log

file until a successful log rotation. If logrotate is configured to compress the log file and delete it, the server may lose the messages logged in this time frame. To avoid this issue, you can configure the logging collector to dynamically assign log file names and use a `prerotate` script to ignore open log files.

Another production-grade approach to managing log output is to send it to syslog and let syslog deal with file rotation. To do this, set the configuration parameter `log_destination` to `syslog` (to log to syslog only) in `postgresql.conf`. Then you can send a `SIGHUP` signal to the syslog daemon whenever you want to force it to start writing a new log file. If you want to automate log rotation, the logrotate program can be configured to work with log files from syslog.

On many systems, however, syslog is not very reliable, particularly with large log messages; it might truncate or drop messages just when you need them the most. Also, on Linux, syslog will flush each message to disk, yielding poor performance. (You can use a “-” at the start of the file name in the syslog configuration file to disable syncing.)

Note that all the solutions described above take care of starting new log files at configurable intervals, but they do not handle deletion of old, no-longer-useful log files. You will probably want to set up a batch job to periodically delete old log files. Another possibility is to configure the rotation program so that old log files are overwritten cyclically.

pgBadger² is an external project that does sophisticated log file analysis. `check_postgres`³ provides Nagios alerts when important messages appear in the log files, as well as detection of many other extraordinary conditions.

² <https://pgbadger.darold.net/>

³ https://bucardo.org/check_postgres/

Chapter 25. Backup and Restore

As with everything that contains valuable data, PostgreSQL databases should be backed up regularly. While the procedure is essentially simple, it is important to have a clear understanding of the underlying techniques and assumptions.

There are three fundamentally different approaches to backing up PostgreSQL data:

- SQL dump
- File system level backup
- Continuous archiving

Each has its own strengths and weaknesses; each is discussed in turn in the following sections.

25.1. SQL Dump

The idea behind this dump method is to generate a file with SQL commands that, when fed back to the server, will recreate the database in the same state as it was at the time of the dump. PostgreSQL provides the utility program `pg_dump` for this purpose. The basic usage of this command is:

```
pg_dump dbname > dumpfile
```

As you see, `pg_dump` writes its result to the standard output. We will see below how this can be useful. While the above command creates a text file, `pg_dump` can create files in other formats that allow for parallelism and more fine-grained control of object restoration.

`pg_dump` is a regular PostgreSQL client application (albeit a particularly clever one). This means that you can perform this backup procedure from any remote host that has access to the database. But remember that `pg_dump` does not operate with special permissions. In particular, it must have read access to all tables that you want to back up, so in order to back up the entire database you almost always have to run it as a database superuser. (If you do not have sufficient privileges to back up the entire database, you can still back up portions of the database to which you do have access using options such as `-n schema` or `-t table`.)

To specify which database server `pg_dump` should contact, use the command line options `-h host` and `-p port`. The default host is the local host or whatever your `PGHOST` environment variable specifies. Similarly, the default port is indicated by the `PGPORT` environment variable or, failing that, by the compiled-in default. (Conveniently, the server will normally have the same compiled-in default.)

Like any other PostgreSQL client application, `pg_dump` will by default connect with the database user name that is equal to the current operating system user name. To override this, either specify the `-U` option or set the environment variable `PGUSER`. Remember that `pg_dump` connections are subject to the normal client authentication mechanisms (which are described in Chapter 20).

An important advantage of `pg_dump` over the other backup methods described later is that `pg_dump`'s output can generally be re-loaded into newer versions of PostgreSQL, whereas file-level backups and continuous archiving are both extremely server-version-specific. `pg_dump` is also the only method that will work when transferring a database to a different machine architecture, such as going from a 32-bit to a 64-bit server.

Dumps created by `pg_dump` are internally consistent, meaning, the dump represents a snapshot of the database at the time `pg_dump` began running. `pg_dump` does not block other operations on the database while it is working. (Exceptions are those operations that need to operate with an exclusive lock, such as most forms of `ALTER TABLE`.)

25.1.1. Restoring the Dump

Text files created by `pg_dump` are intended to be read by the `psql` program using its default settings. The general command form to restore a text dump is

```
psql -X dbname < dumpfile
```

where *dumpfile* is the file output by the `pg_dump` command. The database *dbname* will not be created by this command, so you must create it yourself from `template0` before executing `psql` (e.g., with `createdb -T template0 dbname`). To ensure `psql` runs with its default settings, use the `-X` (`--no-psqlrc`) option. `psql` supports options similar to `pg_dump` for specifying the database server to connect to and the user name to use. See the `psql` reference page for more information.

Non-text file dumps should be restored using the `pg_restore` utility.

Before restoring an SQL dump, all the users who own objects or were granted permissions on objects in the dumped database must already exist. If they do not, the restore will fail to recreate the objects with the original ownership and/or permissions. (Sometimes this is what you want, but usually it is not.)

By default, the `psql` script will continue to execute after an SQL error is encountered. You might wish to run `psql` with the `ON_ERROR_STOP` variable set to alter that behavior and have `psql` exit with an exit status of 3 if an SQL error occurs:

```
psql -X --set ON_ERROR_STOP=on dbname < dumpfile
```

Either way, you will only have a partially restored database. Alternatively, you can specify that the whole dump should be restored as a single transaction, so the restore is either fully completed or fully rolled back. This mode can be specified by passing the `-1` or `--single-transaction` command-line options to `psql`. When using this mode, be aware that even a minor error can rollback a restore that has already run for many hours. However, that might still be preferable to manually cleaning up a complex database after a partially restored dump.

The ability of `pg_dump` and `psql` to write to or read from pipes makes it possible to dump a database directly from one server to another, for example:

```
pg_dump -h host1 dbname | psql -X -h host2 dbname
```

Important

The dumps produced by `pg_dump` are relative to `template0`. This means that any languages, procedures, etc. added via `template1` will also be dumped by `pg_dump`. As a result, when restoring, if you are using a customized `template1`, you must create the empty database from `template0`, as in the example above.

After restoring a backup, it is wise to run `ANALYZE` on each database so the query optimizer has useful statistics; see Section 24.1.3 and Section 24.1.6 for more information. For more advice on how to load large amounts of data into PostgreSQL efficiently, refer to Section 14.4.

25.1.2. Using `pg_dumpall`

`pg_dump` dumps only a single database at a time, and it does not dump information about roles or tablespaces (because those are cluster-wide rather than per-database). To support convenient dumping of the entire contents of a database cluster, the `pg_dumpall` program is provided. `pg_dumpall` backs

up each database in a given cluster, and also preserves cluster-wide data such as role and tablespace definitions. The basic usage of this command is:

```
pg_dumpall > dumpfile
```

The resulting dump can be restored with `psql`:

```
psql -X -f dumpfile postgres
```

(Actually, you can specify any existing database name to start from, but if you are loading into an empty cluster then `postgres` should usually be used.) It is always necessary to have database superuser access when restoring a `pg_dumpall` dump, as that is required to restore the role and tablespace information. If you use tablespaces, make sure that the tablespace paths in the dump are appropriate for the new installation.

`pg_dumpall` works by emitting commands to re-create roles, tablespaces, and empty databases, then invoking `pg_dump` for each database. This means that while each database will be internally consistent, the snapshots of different databases are not synchronized.

Cluster-wide data can be dumped alone using the `pg_dumpall --globals-only` option. This is necessary to fully backup the cluster if running the `pg_dump` command on individual databases.

25.1.3. Handling Large Databases

Some operating systems have maximum file size limits that cause problems when creating large `pg_dump` output files. Fortunately, `pg_dump` can write to the standard output, so you can use standard Unix tools to work around this potential problem. There are several possible methods:

Use compressed dumps. You can use your favorite compression program, for example `gzip`:

```
pg_dump dbname | gzip > filename.gz
```

Reload with:

```
gunzip -c filename.gz | psql dbname
```

or:

```
cat filename.gz | gunzip | psql dbname
```

Use `split`. The `split` command allows you to split the output into smaller files that are acceptable in size to the underlying file system. For example, to make 2 gigabyte chunks:

```
pg_dump dbname | split -b 2G - filename
```

Reload with:

```
cat filename* | psql dbname
```

If using GNU `split`, it is possible to use it and `gzip` together:

```
pg_dump dbname | split -b 2G --filter='gzip > $FILE.gz'
```

It can be restored using `zcat`.

Use `pg_dump`'s custom dump format. If PostgreSQL was built on a system with the `zlib` compression library installed, the custom dump format will compress data as it writes it to the output file. This will produce dump file sizes similar to using `gzip`, but it has the added advantage that tables can be restored selectively. The following command dumps a database using the custom dump format:

```
pg_dump -Fc dbname > filename
```

A custom-format dump is not a script for `psql`, but instead must be restored with `pg_restore`, for example:

```
pg_restore -d dbname filename
```

See the `pg_dump` and `pg_restore` reference pages for details.

For very large databases, you might need to combine `split` with one of the other two approaches.

Use `pg_dump`'s parallel dump feature. To speed up the dump of a large database, you can use `pg_dump`'s parallel mode. This will dump multiple tables at the same time. You can control the degree of parallelism with the `-j` parameter. Parallel dumps are only supported for the "directory" archive format.

```
pg_dump -j num -F d -f out.dir dbname
```

You can use `pg_restore -j` to restore a dump in parallel. This will work for any archive of either the "custom" or the "directory" archive mode, whether or not it has been created with `pg_dump -j`.

25.2. File System Level Backup

An alternative backup strategy is to directly copy the files that PostgreSQL uses to store the data in the database; Section 18.2 explains where these files are located. You can use whatever method you prefer for doing file system backups; for example:

```
tar -cf backup.tar /usr/local/pgsql/data
```

There are two restrictions, however, which make this method impractical, or at least inferior to the `pg_dump` method:

1. The database server *must* be shut down in order to get a usable backup. Half-way measures such as disallowing all connections will *not* work (in part because `tar` and similar tools do not take an atomic snapshot of the state of the file system, but also because of internal buffering within the server). Information about stopping the server can be found in Section 18.5. Needless to say, you also need to shut down the server before restoring the data.
2. If you have dug into the details of the file system layout of the database, you might be tempted to try to back up or restore only certain individual tables or databases from their respective files or directories. This will *not* work because the information contained in these files is not usable without the commit log files, `pg_xact/*`, which contain the commit status of all transactions. A table file is only usable with this information. Of course it is also impossible to restore only a table and the associated `pg_xact` data because that would render all other tables in the database cluster useless. So file system backups only work for complete backup and restoration of an entire database cluster.

An alternative file-system backup approach is to make a “consistent snapshot” of the data directory, if the file system supports that functionality (and you are willing to trust that it is implemented correctly). The typical procedure is to make a “frozen snapshot” of the volume containing the database, then copy the whole data directory (not just parts, see above) from the snapshot to a backup device, then release the frozen snapshot. This will work even while the database server is running. However, a backup created in this way saves the database files in a state as if the database server was not properly shut down; therefore, when you start the database server on the backed-up data, it will think the previous server instance crashed and will replay the WAL log. This is not a problem; just be aware of it (and be sure to include the WAL files in your backup). You can perform a `CHECKPOINT` before taking the snapshot to reduce recovery time.

If your database is spread across multiple file systems, there might not be any way to obtain exactly-simultaneous frozen snapshots of all the volumes. For example, if your data files and WAL log are on different disks, or if tablespaces are on different file systems, it might not be possible to use snapshot backup because the snapshots *must* be simultaneous. Read your file system documentation very carefully before trusting the consistent-snapshot technique in such situations.

If simultaneous snapshots are not possible, one option is to shut down the database server long enough to establish all the frozen snapshots. Another option is to perform a continuous archiving base backup (Section 25.3.2) because such backups are immune to file system changes during the backup. This requires enabling continuous archiving just during the backup process; restore is done using continuous archive recovery (Section 25.3.5).

Another option is to use `rsync` to perform a file system backup. This is done by first running `rsync` while the database server is running, then shutting down the database server long enough to do an `rsync --checksum`. (`--checksum` is necessary because `rsync` only has file modification-time granularity of one second.) The second `rsync` will be quicker than the first, because it has relatively little data to transfer, and the end result will be consistent because the server was down. This method allows a file system backup to be performed with minimal downtime.

Note that a file system backup will typically be larger than an SQL dump. (`pg_dump` does not need to dump the contents of indexes for example, just the commands to recreate them.) However, taking a file system backup might be faster.

25.3. Continuous Archiving and Point-in-Time Recovery (PITR)

At all times, PostgreSQL maintains a *write ahead log* (WAL) in the `pg_wal/` subdirectory of the cluster's data directory. The log records every change made to the database's data files. This log exists primarily for crash-safety purposes: if the system crashes, the database can be restored to consistency by “replaying” the log entries made since the last checkpoint. However, the existence of the log makes it possible to use a third strategy for backing up databases: we can combine a file-system-level backup with backup of the WAL files. If recovery is needed, we restore the file system backup and then replay from the backed-up WAL files to bring the system to a current state. This approach is more complex to administer than either of the previous approaches, but it has some significant benefits:

- We do not need a perfectly consistent file system backup as the starting point. Any internal inconsistency in the backup will be corrected by log replay (this is not significantly different from what happens during crash recovery). So we do not need a file system snapshot capability, just `tar` or a similar archiving tool.
- Since we can combine an indefinitely long sequence of WAL files for replay, continuous backup can be achieved simply by continuing to archive the WAL files. This is particularly valuable for large databases, where it might not be convenient to take a full backup frequently.
- It is not necessary to replay the WAL entries all the way to the end. We could stop the replay at any point and have a consistent snapshot of the database as it was at that time. Thus, this technique

supports *point-in-time recovery*: it is possible to restore the database to its state at any time since your base backup was taken.

- If we continuously feed the series of WAL files to another machine that has been loaded with the same base backup file, we have a *warm standby* system: at any point we can bring up the second machine and it will have a nearly-current copy of the database.

Note

`pg_dump` and `pg_dumpall` do not produce file-system-level backups and cannot be used as part of a continuous-archiving solution. Such dumps are *logical* and do not contain enough information to be used by WAL replay.

As with the plain file-system-backup technique, this method can only support restoration of an entire database cluster, not a subset. Also, it requires a lot of archival storage: the base backup might be bulky, and a busy system will generate many megabytes of WAL traffic that have to be archived. Still, it is the preferred backup technique in many situations where high reliability is needed.

To recover successfully using continuous archiving (also called “online backup” by many database vendors), you need a continuous sequence of archived WAL files that extends back at least as far as the start time of your backup. So to get started, you should set up and test your procedure for archiving WAL files *before* you take your first base backup. Accordingly, we first discuss the mechanics of archiving WAL files.

25.3.1. Setting Up WAL Archiving

In an abstract sense, a running PostgreSQL system produces an indefinitely long sequence of WAL records. The system physically divides this sequence into WAL *segment files*, which are normally 16MB apiece (although the segment size can be altered during `initdb`). The segment files are given numeric names that reflect their position in the abstract WAL sequence. When not using WAL archiving, the system normally creates just a few segment files and then “recycles” them by renaming no-longer-needed segment files to higher segment numbers. It’s assumed that segment files whose contents precede the last checkpoint are no longer of interest and can be recycled.

When archiving WAL data, we need to capture the contents of each segment file once it is filled, and save that data somewhere before the segment file is recycled for reuse. Depending on the application and the available hardware, there could be many different ways of “saving the data somewhere”: we could copy the segment files to an NFS-mounted directory on another machine, write them onto a tape drive (ensuring that you have a way of identifying the original name of each file), or batch them together and burn them onto CDs, or something else entirely. To provide the database administrator with flexibility, PostgreSQL tries not to make any assumptions about how the archiving will be done. Instead, PostgreSQL lets the administrator specify a shell command or an archive library to be executed to copy a completed segment file to wherever it needs to go. This could be as simple as a shell command that uses `cp`, or it could invoke a complex C function — it’s all up to you.

To enable WAL archiving, set the `wal_level` configuration parameter to `replica` or higher, `archive_mode` to `on`, specify the shell command to use in the `archive_command` configuration parameter or specify the library to use in the `archive_library` configuration parameter. In practice these settings will always be placed in the `postgresql.conf` file.

In `archive_command`, `%p` is replaced by the path name of the file to archive, while `%f` is replaced by only the file name. (The path name is relative to the current working directory, i.e., the cluster’s data directory.) Use `%%` if you need to embed an actual `%` character in the command. The simplest useful command is something like:

```
archive_command = 'test ! -f /mnt/server/archivedir/%f && cp %p /
mnt/server/archivedir/%f' # Unix
archive_command = 'copy "%p" "C:\\server\\archivedir\\%f"' #
Windows
```

which will copy archivable WAL segments to the directory `/mnt/server/archivedir`. (This is an example, not a recommendation, and might not work on all platforms.) After the `%p` and `%f` parameters have been replaced, the actual command executed might look like this:

```
test ! -f /mnt/server/archivedir/00000001000000A9000000065
&& cp pg_wal/00000001000000A9000000065 /mnt/server/
archivedir/00000001000000A9000000065
```

A similar command will be generated for each new file to be archived.

The archive command will be executed under the ownership of the same user that the PostgreSQL server is running as. Since the series of WAL files being archived contains effectively everything in your database, you will want to be sure that the archived data is protected from prying eyes; for example, archive into a directory that does not have group or world read access.

It is important that the archive command return zero exit status if and only if it succeeds. Upon getting a zero result, PostgreSQL will assume that the file has been successfully archived, and will remove or recycle it. However, a nonzero status tells PostgreSQL that the file was not archived; it will try again periodically until it succeeds.

Another way to archive is to use a custom archive module as the `archive_library`. Since such modules are written in C, creating your own may require considerably more effort than writing a shell command. However, archive modules can be more performant than archiving via shell, and they will have access to many useful server resources. For more information about archive modules, see Chapter 49.

When the archive command is terminated by a signal (other than `SIGTERM` that is used as part of a server shutdown) or an error by the shell with an exit status greater than 125 (such as command not found), or if the archive function emits an `ERROR` or `FATAL`, the archiver process aborts and gets restarted by the postmaster. In such cases, the failure is not reported in `pg_stat_archiver`.

Archive commands and libraries should generally be designed to refuse to overwrite any pre-existing archive file. This is an important safety feature to preserve the integrity of your archive in case of administrator error (such as sending the output of two different servers to the same archive directory). It is advisable to test your proposed archive library to ensure that it does not overwrite an existing file.

In rare cases, PostgreSQL may attempt to re-archive a WAL file that was previously archived. For example, if the system crashes before the server makes a durable record of archival success, the server will attempt to archive the file again after restarting (provided archiving is still enabled). When an archive command or library encounters a pre-existing file, it should return a zero status or `true`, respectively, if the WAL file has identical contents to the pre-existing archive and the pre-existing archive is fully persisted to storage. If a pre-existing file contains different contents than the WAL file being archived, the archive command or library *must* return a nonzero status or `false`, respectively.

The example command above for Unix avoids overwriting a pre-existing archive by including a separate `test` step. On some Unix platforms, `cp` has switches such as `-i` that can be used to do the same thing less verbosely, but you should not rely on these without verifying that the right exit status is returned. (In particular, GNU `cp` will return status zero when `-i` is used and the target file already exists, which is *not* the desired behavior.)

While designing your archiving setup, consider what will happen if the archive command or library fails repeatedly because some aspect requires operator intervention or the archive runs out of space. For example, this could occur if you write to tape without an autochanger; when the tape fills, nothing further can be archived until the tape is swapped. You should ensure that any error condition or request

to a human operator is reported appropriately so that the situation can be resolved reasonably quickly. The `pg_wal/` directory will continue to fill with WAL segment files until the situation is resolved. (If the file system containing `pg_wal/` fills up, PostgreSQL will do a PANIC shutdown. No committed transactions will be lost, but the database will remain offline until you free some space.)

The speed of the archive command or library is unimportant as long as it can keep up with the average rate at which your server generates WAL data. Normal operation continues even if the archiving process falls a little behind. If archiving falls significantly behind, this will increase the amount of data that would be lost in the event of a disaster. It will also mean that the `pg_wal/` directory will contain large numbers of not-yet-archived segment files, which could eventually exceed available disk space. You are advised to monitor the archiving process to ensure that it is working as you intend.

In writing your archive command or library, you should assume that the file names to be archived can be up to 64 characters long and can contain any combination of ASCII letters, digits, and dots. It is not necessary to preserve the original relative path (`%p`) but it is necessary to preserve the file name (`%f`).

Note that although WAL archiving will allow you to restore any modifications made to the data in your PostgreSQL database, it will not restore changes made to configuration files (that is, `postgresql.conf`, `pg_hba.conf` and `pg_ident.conf`), since those are edited manually rather than through SQL operations. You might wish to keep the configuration files in a location that will be backed up by your regular file system backup procedures. See Section 19.2 for how to relocate the configuration files.

The archive command or function is only invoked on completed WAL segments. Hence, if your server generates only little WAL traffic (or has slack periods where it does so), there could be a long delay between the completion of a transaction and its safe recording in archive storage. To put a limit on how old unarchived data can be, you can set `archive_timeout` to force the server to switch to a new WAL segment file at least that often. Note that archived files that are archived early due to a forced switch are still the same length as completely full files. It is therefore unwise to set a very short `archive_timeout` — it will bloat your archive storage. `archive_timeout` settings of a minute or so are usually reasonable.

Also, you can force a segment switch manually with `pg_switch_wal` if you want to ensure that a just-finished transaction is archived as soon as possible. Other utility functions related to WAL management are listed in Table 9.95.

When `wal_level` is `minimal` some SQL commands are optimized to avoid WAL logging, as described in Section 14.4.7. If archiving or streaming replication were turned on during execution of one of these statements, WAL would not contain enough information for archive recovery. (Crash recovery is unaffected.) For this reason, `wal_level` can only be changed at server start. However, `archive_command` and `archive_library` can be changed with a configuration file reload. If you are archiving via shell and wish to temporarily stop archiving, one way to do it is to set `archive_command` to the empty string (`' '`). This will cause WAL files to accumulate in `pg_wal/` until a working `archive_command` is re-established.

25.3.2. Making a Base Backup

The easiest way to perform a base backup is to use the `pg_basebackup` tool. It can create a base backup either as regular files or as a tar archive. If more flexibility than `pg_basebackup` can provide is required, you can also make a base backup using the low level API (see Section 25.3.4).

It is not necessary to be concerned about the amount of time it takes to make a base backup. However, if you normally run the server with `full_page_writes` disabled, you might notice a drop in performance while the backup runs since `full_page_writes` is effectively forced on during backup mode.

To make use of the backup, you will need to keep all the WAL segment files generated during and after the file system backup. To aid you in doing this, the base backup process creates a *backup history file* that is immediately stored into the WAL archive area. This file is named after the first WAL segment file that you need for the file system backup. For example, if the starting WAL

file is 0000000100001234000055CD the backup history file will be named something like 0000000100001234000055CD.007C9330.backup. (The second part of the file name stands for an exact position within the WAL file, and can ordinarily be ignored.) Once you have safely archived the file system backup and the WAL segment files used during the backup (as specified in the backup history file), all archived WAL segments with names numerically less are no longer needed to recover the file system backup and can be deleted. However, you should consider keeping several backup sets to be absolutely certain that you can recover your data.

The backup history file is just a small text file. It contains the label string you gave to `pg_basebackup`, as well as the starting and ending times and WAL segments of the backup. If you used the label to identify the associated dump file, then the archived history file is enough to tell you which dump file to restore.

Since you have to keep around all the archived WAL files back to your last base backup, the interval between base backups should usually be chosen based on how much storage you want to expend on archived WAL files. You should also consider how long you are prepared to spend recovering, if recovery should be necessary — the system will have to replay all those WAL segments, and that could take awhile if it has been a long time since the last base backup.

25.3.3. Making an Incremental Backup

You can use `pg_basebackup` to take an incremental backup by specifying the `--incremental` option. You must supply, as an argument to `--incremental`, the backup manifest to an earlier backup from the same server. In the resulting backup, non-relation files will be included in their entirety, but some relation files may be replaced by smaller incremental files which contain only the blocks which have been changed since the earlier backup and enough metadata to reconstruct the current version of the file.

To figure out which blocks need to be backed up, the server uses WAL summaries, which are stored in the data directory, inside the directory `pg_wal/summaries`. If the required summary files are not present, an attempt to take an incremental backup will fail. The summaries present in this directory must cover all LSNs from the start LSN of the prior backup to the start LSN of the current backup. Since the server looks for WAL summaries just after establishing the start LSN of the current backup, the necessary summary files probably won't be instantly present on disk, but the server will wait for any missing files to show up. This also helps if the WAL summarization process has fallen behind. However, if the necessary files have already been removed, or if the WAL summarizer doesn't catch up quickly enough, the incremental backup will fail.

When restoring an incremental backup, it will be necessary to have not only the incremental backup itself but also all earlier backups that are required to supply the blocks omitted from the incremental backup. See `pg_combinebackup` for further information about this requirement. Note that there are restrictions on the use of `pg_combinebackup` when the checksum status of the cluster has been changed; see `pg_combinebackup` limitations.

Note that all of the requirements for making use of a full backup also apply to an incremental backup. For instance, you still need all of the WAL segment files generated during and after the file system backup, and any relevant WAL history files. And you still need to create a `recovery.signal` (or `standby.signal`) and perform recovery, as described in Section 25.3.5. The requirement to have earlier backups available at restore time and to use `pg_combinebackup` is an additional requirement on top of everything else. Keep in mind that PostgreSQL has no built-in mechanism to figure out which backups are still needed as a basis for restoring later incremental backups. You must keep track of the relationships between your full and incremental backups on your own, and be certain not to remove earlier backups if they might be needed when restoring later incremental backups.

Incremental backups typically only make sense for relatively large databases where a significant portion of the data does not change, or only changes slowly. For a small database, it's simpler to ignore the existence of incremental backups and simply take full backups, which are simpler to manage. For a large database all of which is heavily modified, incremental backups won't be much smaller than full backups.

An incremental backup is only possible if replay would begin from a later checkpoint than for the previous backup upon which it depends. If you take the incremental backup on the primary, this condition is always satisfied, because each backup triggers a new checkpoint. On a standby, replay begins from the most recent restartpoint. Therefore, an incremental backup of a standby server can fail if there has been very little activity since the previous backup, since no new restartpoint might have been created.

25.3.4. Making a Base Backup Using the Low Level API

Instead of taking a full or incremental base backup using `pg_basebackup`, you can take a base backup using the low-level API. This procedure contains a few more steps than the `pg_basebackup` method, but is relatively simple. It is very important that these steps are executed in sequence, and that the success of a step is verified before proceeding to the next step.

Multiple backups are able to be run concurrently (both those started using this backup API and those started using `pg_basebackup`).

1. Ensure that WAL archiving is enabled and working.
2. Connect to the server (it does not matter which database) as a user with rights to run `pg_backup_start` (superuser, or a user who has been granted `EXECUTE` on the function) and issue the command:

```
SELECT pg_backup_start(label => 'label', fast => false);
```

where `label` is any string you want to use to uniquely identify this backup operation. The connection calling `pg_backup_start` must be maintained until the end of the backup, or the backup will be automatically aborted.

Online backups are always started at the beginning of a checkpoint. By default, `pg_backup_start` will wait for the next regularly scheduled checkpoint to complete, which may take a long time (see the configuration parameters `checkpoint_timeout` and `checkpoint_completion_target`). This is usually preferable as it minimizes the impact on the running system. If you want to start the backup as soon as possible, pass `true` as the second parameter to `pg_backup_start` and it will request an immediate checkpoint, which will finish as fast as possible using as much I/O as possible.

3. Perform the backup, using any convenient file-system-backup tool such as `tar` or `cpio` (not `pg_dump` or `pg_dumpall`). It is neither necessary nor desirable to stop normal operation of the database while you do this. See Section 25.3.4.1 for things to consider during this backup.
4. In the same connection as before, issue the command:

```
SELECT * FROM pg_backup_stop(wait_for_archive => true);
```

This terminates backup mode. On a primary, it also performs an automatic switch to the next WAL segment. On a standby, it is not possible to automatically switch WAL segments, so you may wish to run `pg_switch_wal` on the primary to perform a manual switch. The reason for the switch is to arrange for the last WAL segment file written during the backup interval to be ready to archive.

`pg_backup_stop` will return one row with three values. The second of these fields should be written to a file named `backup_label` in the root directory of the backup. The third field should be written to a file named `tablespace_map` unless the field is empty. These files are vital to the backup working and must be written byte for byte without modification, which may require opening the file in binary mode.

5. Once the WAL segment files active during the backup are archived, you are done. The file identified by `pg_backup_stop`'s first return value is the last segment that is required to

form a complete set of backup files. On a primary, if `archive_mode` is enabled and the `wait_for_archive` parameter is `true`, `pg_backup_stop` does not return until the last segment has been archived. On a standby, `archive_mode` must be `always` in order for `pg_backup_stop` to wait. Archiving of these files happens automatically since you have already configured `archive_command` or `archive_library`. In most cases this happens quickly, but you are advised to monitor your archive system to ensure there are no delays. If the archive process has fallen behind because of failures of the archive command or library, it will keep retrying until the archive succeeds and the backup is complete. If you wish to place a time limit on the execution of `pg_backup_stop`, set an appropriate `statement_timeout` value, but make note that if `pg_backup_stop` terminates because of this your backup may not be valid.

If the backup process monitors and ensures that all WAL segment files required for the backup are successfully archived then the `wait_for_archive` parameter (which defaults to `true`) can be set to `false` to have `pg_backup_stop` return as soon as the stop backup record is written to the WAL. By default, `pg_backup_stop` will wait until all WAL has been archived, which can take some time. This option must be used with caution: if WAL archiving is not monitored correctly then the backup might not include all of the WAL files and will therefore be incomplete and not able to be restored.

25.3.4.1. Backing Up the Data Directory

Some file system backup tools emit warnings or errors if the files they are trying to copy change while the copy proceeds. When taking a base backup of an active database, this situation is normal and not an error. However, you need to ensure that you can distinguish complaints of this sort from real errors. For example, some versions of `rsync` return a separate exit code for “vanished source files”, and you can write a driver script to accept this exit code as a non-error case. Also, some versions of GNU `tar` return an error code indistinguishable from a fatal error if a file was truncated while `tar` was copying it. Fortunately, GNU `tar` versions 1.16 and later exit with 1 if a file was changed during the backup, and 2 for other errors. With GNU `tar` version 1.23 and later, you can use the warning options `--warning=no-file-changed` `--warning=no-file-removed` to hide the related warning messages.

Be certain that your backup includes all of the files under the database cluster directory (e.g., `/usr/local/pgsql/data`). If you are using tablespaces that do not reside underneath this directory, be careful to include them as well (and be sure that your backup archives symbolic links as links, otherwise the restore will corrupt your tablespaces).

You should, however, omit from the backup the files within the cluster's `pg_wal/` subdirectory. This slight adjustment is worthwhile because it reduces the risk of mistakes when restoring. This is easy to arrange if `pg_wal/` is a symbolic link pointing to someplace outside the cluster directory, which is a common setup anyway for performance reasons. You might also want to exclude `postmaster.pid` and `postmaster.opts`, which record information about the running postmaster, not about the postmaster which will eventually use this backup. (These files can confuse `pg_ctl`.)

It is often a good idea to also omit from the backup the files within the cluster's `pg_repslot/` directory, so that replication slots that exist on the primary do not become part of the backup. Otherwise, the subsequent use of the backup to create a standby may result in indefinite retention of WAL files on the standby, and possibly bloat on the primary if hot standby feedback is enabled, because the clients that are using those replication slots will still be connecting to and updating the slots on the primary, not the standby. Even if the backup is only intended for use in creating a new primary, copying the replication slots isn't expected to be particularly useful, since the contents of those slots will likely be badly out of date by the time the new primary comes on line.

The contents of the directories `pg_dynshmem/`, `pg_notify/`, `pg_serial/`, `pg_snapshots/`, `pg_stat_tmp/`, and `pg_subtrans/` (but not the directories themselves) can be omitted from the backup as they will be initialized on postmaster startup.

Any file or directory beginning with `pgsql_tmp` can be omitted from the backup. These files are removed on postmaster start and the directories will be recreated as needed.

`pg_internal.init` files can be omitted from the backup whenever a file of that name is found. These files contain relation cache data that is always rebuilt when recovering.

The backup label file includes the label string you gave to `pg_backup_start`, as well as the time at which `pg_backup_start` was run, and the name of the starting WAL file. In case of confusion it is therefore possible to look inside a backup file and determine exactly which backup session the dump file came from. The tablespace map file includes the symbolic link names as they exist in the directory `pg_tblspc/` and the full path of each symbolic link. These files are not merely for your information; their presence and contents are critical to the proper operation of the system's recovery process.

It is also possible to make a backup while the server is stopped. In this case, you obviously cannot use `pg_backup_start` or `pg_backup_stop`, and you will therefore be left to your own devices to keep track of which backup is which and how far back the associated WAL files go. It is generally better to follow the continuous archiving procedure above.

25.3.5. Recovering Using a Continuous Archive Backup

Okay, the worst has happened and you need to recover from your backup. Here is the procedure:

1. Stop the server, if it's running.
2. If you have the space to do so, copy the whole cluster data directory and any tablespaces to a temporary location in case you need them later. Note that this precaution will require that you have enough free space on your system to hold two copies of your existing database. If you do not have enough space, you should at least save the contents of the cluster's `pg_wal` subdirectory, as it might contain WAL files which were not archived before the system went down.
3. Remove all existing files and subdirectories under the cluster data directory and under the root directories of any tablespaces you are using.
4. If you're restoring a full backup, you can restore the database files directly into the target directories. Be sure that they are restored with the right ownership (the database system user, not `root`!) and with the right permissions. If you are using tablespaces, you should verify that the symbolic links in `pg_tblspc/` were correctly restored.
5. If you're restoring an incremental backup, you'll need to restore the incremental backup and all earlier backups upon which it directly or indirectly depends to the machine where you are performing the restore. These backups will need to be placed in separate directories, not the target directories where you want the running server to end up. Once this is done, use `pg_combinebackup` to pull data from the full backup and all of the subsequent incremental backups and write out a synthetic full backup to the target directories. As above, verify that permissions and tablespace links are correct.
6. Remove any files present in `pg_wal/`; these came from the file system backup and are therefore probably obsolete rather than current. If you didn't archive `pg_wal/` at all, then recreate it with proper permissions, being careful to ensure that you re-establish it as a symbolic link if you had it set up that way before.
7. If you have unarchived WAL segment files that you saved in step 2, copy them into `pg_wal/`. (It is best to copy them, not move them, so you still have the unmodified files if a problem occurs and you have to start over.)
8. Set recovery configuration settings in `postgresql.conf` (see Section 19.5.5) and create a file `recovery.signal` in the cluster data directory. You might also want to temporarily modify `pg_hba.conf` to prevent ordinary users from connecting until you are sure the recovery was successful.
9. Start the server. The server will go into recovery mode and proceed to read through the archived WAL files it needs. Should the recovery be terminated because of an external error, the server can simply be restarted and it will continue recovery. Upon completion of the recovery process,

the server will remove `recovery.signal` (to prevent accidentally re-entering recovery mode later) and then commence normal database operations.

10. Inspect the contents of the database to ensure you have recovered to the desired state. If not, return to step 1. If all is well, allow your users to connect by restoring `pg_hba.conf` to normal.

The key part of all this is to set up a recovery configuration that describes how you want to recover and how far the recovery should run. The one thing that you absolutely must specify is the `restore_command`, which tells PostgreSQL how to retrieve archived WAL file segments. Like the `archive_command`, this is a shell command string. It can contain `%f`, which is replaced by the name of the desired WAL file, and `%p`, which is replaced by the path name to copy the WAL file to. (The path name is relative to the current working directory, i.e., the cluster's data directory.) Write `%%` if you need to embed an actual `%` character in the command. The simplest useful command is something like:

```
restore_command = 'cp /mnt/server/archivedir/%f %p'
```

which will copy previously archived WAL segments from the directory `/mnt/server/archivedir`. Of course, you can use something much more complicated, perhaps even a shell script that requests the operator to mount an appropriate tape.

It is important that the command return nonzero exit status on failure. The command *will* be called requesting files that are not present in the archive; it must return nonzero when so asked. This is not an error condition. An exception is that if the command was terminated by a signal (other than `SIGTERM`, which is used as part of a database server shutdown) or an error by the shell (such as command not found), then recovery will abort and the server will not start up.

Not all of the requested files will be WAL segment files; you should also expect requests for files with a suffix of `.history`. Also be aware that the base name of the `%p` path will be different from `%f`; do not expect them to be interchangeable.

WAL segments that cannot be found in the archive will be sought in `pg_wal/`; this allows use of recent un-archived segments. However, segments that are available from the archive will be used in preference to files in `pg_wal/`.

Normally, recovery will proceed through all available WAL segments, thereby restoring the database to the current point in time (or as close as possible given the available WAL segments). Therefore, a normal recovery will end with a “file not found” message, the exact text of the error message depending upon your choice of `restore_command`. You may also see an error message at the start of recovery for a file named something like `00000001.history`. This is also normal and does not indicate a problem in simple recovery situations; see Section 25.3.6 for discussion.

If you want to recover to some previous point in time (say, right before the junior DBA dropped your main transaction table), just specify the required stopping point. You can specify the stop point, known as the “recovery target”, either by date/time, named restore point or by completion of a specific transaction ID. As of this writing only the date/time and named restore point options are very usable, since there are no tools to help you identify with any accuracy which transaction ID to use.

Note

The stop point must be after the ending time of the base backup, i.e., the end time of `pg_backup_stop`. You cannot use a base backup to recover to a time when that backup was in progress. (To recover to such a time, you must go back to your previous base backup and roll forward from there.)

If recovery finds corrupted WAL data, recovery will halt at that point and the server will not start. In such a case the recovery process could be re-run from the beginning, specifying a “recovery target” before the point of corruption so that recovery can complete normally. If recovery fails for an external

reason, such as a system crash or if the WAL archive has become inaccessible, then the recovery can simply be restarted and it will restart almost from where it failed. Recovery restart works much like checkpointing in normal operation: the server periodically forces all its state to disk, and then updates the `pg_control` file to indicate that the already-processed WAL data need not be scanned again.

25.3.6. Timelines

The ability to restore the database to a previous point in time creates some complexities that are akin to science-fiction stories about time travel and parallel universes. For example, in the original history of the database, suppose you dropped a critical table at 5:15PM on Tuesday evening, but didn't realize your mistake until Wednesday noon. Unfazed, you get out your backup, restore to the point-in-time 5:14PM Tuesday evening, and are up and running. In *this* history of the database universe, you never dropped the table. But suppose you later realize this wasn't such a great idea, and would like to return to sometime Wednesday morning in the original history. You won't be able to if, while your database was up-and-running, it overwrote some of the WAL segment files that led up to the time you now wish you could get back to. Thus, to avoid this, you need to distinguish the series of WAL records generated after you've done a point-in-time recovery from those that were generated in the original database history.

To deal with this problem, PostgreSQL has a notion of *timelines*. Whenever an archive recovery completes, a new timeline is created to identify the series of WAL records generated after that recovery. The timeline ID number is part of WAL segment file names so a new timeline does not overwrite the WAL data generated by previous timelines. For example, in the WAL file name `0000000100001234000055CD`, the leading `00000001` is the timeline ID in hexadecimal. (Note that in other contexts, such as server log messages, timeline IDs are usually printed in decimal.)

It is in fact possible to archive many different timelines. While that might seem like a useless feature, it's often a lifesaver. Consider the situation where you aren't quite sure what point-in-time to recover to, and so have to do several point-in-time recoveries by trial and error until you find the best place to branch off from the old history. Without timelines this process would soon generate an unmanageable mess. With timelines, you can recover to *any* prior state, including states in timeline branches that you abandoned earlier.

Every time a new timeline is created, PostgreSQL creates a “timeline history” file that shows which timeline it branched off from and when. These history files are necessary to allow the system to pick the right WAL segment files when recovering from an archive that contains multiple timelines. Therefore, they are archived into the WAL archive area just like WAL segment files. The history files are just small text files, so it's cheap and appropriate to keep them around indefinitely (unlike the segment files which are large). You can, if you like, add comments to a history file to record your own notes about how and why this particular timeline was created. Such comments will be especially valuable when you have a thicket of different timelines as a result of experimentation.

The default behavior of recovery is to recover to the latest timeline found in the archive. If you wish to recover to the timeline that was current when the base backup was taken or into a specific child timeline (that is, you want to return to some state that was itself generated after a recovery attempt), you need to specify `current` or the target timeline ID in `recovery_target_timeline`. You cannot recover into timelines that branched off earlier than the base backup.

25.3.7. Tips and Examples

Some tips for configuring continuous archiving are given here.

25.3.7.1. Standalone Hot Backups

It is possible to use PostgreSQL's backup facilities to produce standalone hot backups. These are backups that cannot be used for point-in-time recovery, yet are typically much faster to backup and restore than `pg_dump` dumps. (They are also much larger than `pg_dump` dumps, so in some cases the speed advantage might be negated.)

As with base backups, the easiest way to produce a standalone hot backup is to use the `pg_basebackup` tool. If you include the `-X` parameter when calling it, all the write-ahead log required to use the backup will be included in the backup automatically, and no special action is required to restore the backup.

25.3.7.2. Compressed Archive Logs

If archive storage size is a concern, you can use `gzip` to compress the archive files:

```
archive_command = 'gzip < %p > /mnt/server/archivedir/%f.gz'
```

You will then need to use `gunzip` during recovery:

```
restore_command = 'gunzip < /mnt/server/archivedir/%f.gz > %p'
```

25.3.7.3. archive_command Scripts

Many people choose to use scripts to define their `archive_command`, so that their `postgresql.conf` entry looks very simple:

```
archive_command = 'local_backup_script.sh "%p" "%f"'
```

Using a separate script file is advisable any time you want to use more than a single command in the archiving process. This allows all complexity to be managed within the script, which can be written in a popular scripting language such as `bash` or `perl`.

Examples of requirements that might be solved within a script include:

- Copying data to secure off-site data storage
- Batching WAL files so that they are transferred every three hours, rather than one at a time
- Interfacing with other backup and recovery software
- Interfacing with monitoring software to report errors

Tip

When using an `archive_command` script, it's desirable to enable `logging_collector`. Any messages written to `stderr` from the script will then appear in the database server log, allowing complex configurations to be diagnosed easily if they fail.

25.3.8. Caveats

At this writing, there are several limitations of the continuous archiving technique. These will probably be fixed in future releases:

- If a `CREATE DATABASE` command is executed while a base backup is being taken, and then the template database that the `CREATE DATABASE` copied is modified while the base backup is still in progress, it is possible that recovery will cause those modifications to be propagated into the created database as well. This is of course undesirable. To avoid this risk, it is best not to modify any template databases while taking a base backup.
- `CREATE TABLESPACE` commands are WAL-logged with the literal absolute path, and will therefore be replayed as tablespace creations with the same absolute path. This might be undesirable if

the WAL is being replayed on a different machine. It can be dangerous even if the WAL is being replayed on the same machine, but into a new data directory: the replay will still overwrite the contents of the original tablespace. To avoid potential gotchas of this sort, the best practice is to take a new base backup after creating or dropping tablespaces.

It should also be noted that the default WAL format is fairly bulky since it includes many disk page snapshots. These page snapshots are designed to support crash recovery, since we might need to fix partially-written disk pages. Depending on your system hardware and software, the risk of partial writes might be small enough to ignore, in which case you can significantly reduce the total volume of archived WAL files by turning off page snapshots using the `full_page_writes` parameter. (Read the notes and warnings in Chapter 28 before you do so.) Turning off page snapshots does not prevent use of the WAL for PITR operations. An area for future development is to compress archived WAL data by removing unnecessary page copies even when `full_page_writes` is on. In the meantime, administrators might wish to reduce the number of page snapshots included in WAL by increasing the checkpoint interval parameters as much as feasible.